Ders 3 Genişletilmiş Notu: Sayı Sistemleri, İkiye Tümleme, IEEE 754 ve İyi Algoritmanın Ölçütleri

Ders 3 Genişletilmiş Notu: Sayı Sistemleri, İkiye Tümleme, IEEE 754 ve İyi Algoritmanın Ölçütleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. İletişim Türleri: Simplex, Half Duplex, Full Duplex
- 2. Neden Farklı Sayı Sistemleri Kullanıyoruz?
- 3. Basamak Değeri ve Taban Aritmetiği
- 4. İkilikten Sekizlik ve On Altılık Sisteme Geçiş
- 5. Sekizlik ve On Altılıktan İkiliğe Geçiş
- 6. Kesirli Sayıların Dönüşümü
- 7. İşaretli Sayılar ve İşaret Biti
- 8. IEEE 754 Floating Point Gösterimi
- 9. Dört Bitlik Tablo: Sayı Sistemlerini Kısa Yoldan Görmek
- 10. Bire Tümleme ve İkiye Tümleme
- 11. Ara Konu: Programı Modüler Yazmak
- 12. Algoritmanın Tanımına Dönüş
- 13. Üç Temel Yapı
- 14. Yol Tarifi Örneği
- 15. Barcelona Metrosundaki Turnike
- 16. Robota Araba Çizdirme
- 17. Tanımlı Olmayan İşlem Kullanılamaz
- 18. İyi Algoritmanın Ölçütleri
- 19. Ödünleşim (Trade-off)
- 20. İyi Algoritma Nasıl Gelişir?
- 21. 1’den N’e Kadar Toplam
- 22. Problemi Genişletmek: m’den n’e Kadar Toplam
- 23. Çözülmeyi Bekleyen Problemler
- 24. Şifreler, Kaba Kuvvet ve Hash Fonksiyonları
- 25. Genellenebilirlik ve Ders Kitabı

1. İletişim Türleri: Simplex, Half Duplex, Full Duplex

Ders, tek taraflı bir kanal mı yoksa herkesin yazabildiği bir grup mu kullanılmalı sorusuyla başladı. Hoca bu vesileyle bilgisayar ağları dersinde karşımıza çıkacak üç iletişim türünü günlük hayattan örneklerle anlattı:

- **Simplex:** İletişim tek yönlüdür. Televizyon gibi düşünebiliriz.
- **Half duplex:** İki yönlüdür ama aynı anda değildir. Telsiz gibi: Ben konuşurken karşıdaki konuşamaz.
- **Full duplex:** İki taraf aynı anda konuşabilir. Normal bir telefon gibi.

Hoca, WhatsApp gibi uygulamaların temel mantığının soket programlama olduğunu da ekledi. İkinci sınıfta yapısal programlamada soket programlamayı anlattığını, bunun üzerine Java veya

Python gibi bir dil öğrenen birinin rahatlıkla bir mobil uygulama geliştirebileceğini söyledi. Bunun için arkadaki soket programı ile multi-thread mantığını bilmek gerekir.

2. Neden Farklı Sayı Sistemleri Kullanıyoruz?

Geçen derste bilgisayar sistemlerinin nasıl çalıştığını ve hangi elemanlardan oluştuğunu konuşmuştuk. Bütün hafıza ve işlem elemanları 0 ve 1 ile çalışır. Peki bu 0 ve 1'leri günlük hayatta kullanmamız mümkün mü? Hocaya göre pek mümkün değil. Bir komut düşünelim: Op kodu 0 ve 1'lerle kodlanmış, yan yana 16 ya da 32 tane bit görünüyor. Buradan bunun toplama komutu mu yoksa başka bir komut mu olduğunu anlamak kolay değildir. Komut 64 bit olursa anlamlandırmak daha da zorlaşır. Aynı durum bir registerda veya bellekte saklanan herhangi bir değer için de geçerlidir.

Bu yüzden ikilik sistemin yanında başka sayı sistemleri kullanırız:

Sistem	Taban	Rakamlar
İkilik (binary)	2	0, 1
Sekizlik (oktal)	8	0-7
Onluk (desimal)	10	0-9
On altılık (heksadesimal)	16	0-9, A, B, C, D, E, F

Hangi sistemde olduğumuzu belirtmek için sayının altına taban yazılır. Onluk sistemde genelde hiçbir şey yazılmaz. On altılık sistemde H ya da 16 yazılabilir.

Bir sayı yalnız bu sistemlere değil, taban aritmetiğine uyan her sisteme dönüştürülebilir. Örneğin yedilik bir sisteme de çevrilebilir.

3. Basamak Değeri ve Taban Aritmetiği

Hoca, daha önce taban aritmetiği görmemiş olanlar için konuyu ilkokuldan bildiğimiz basamak değerine bağladı. 385 sayısı onluk sistemde 3, 8 ve 5'ten oluşur. Birler basamağında 5, onlar basamağında 8, yüzler basamağında 3 vardır:

$$385 = 3 \cdot 10^2 + 8 \cdot 10^1 + 5 \cdot 10^0 = 300 + 80 + 5$$

Aynı mantık diğer sistemlerde de geçerlidir. Hangi tabandaysak o tabanın kuvvetlerini kullanırız. İkilik sistemde basamaklar birler, ikiler, dörtler, sekizler diye gider. Örneğin:

$$(111)_2 = 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 4 + 2 + 1 = 7$$

Soru: 2 üzeri 0 ile mi başlayacağım, 1 ile mi?

Cevap: 2 üzeri 0 ile. Birler basamağı 2^0'lardır. Yedilik sayma sisteminde olsaydık 7^0 ile başlayacaktık.

Hoca lisede anlatılan bir kısa yolu da eleştirdi. Sekizlik sistemde 1276 gibi bir sayı için "1 ile başla, 1 ile 8'i çarp, sonra onu yine 8 ile çarp" denir. Hocaya göre bu çarpmalara gerek yoktur. Kuvvetler doğrudan $8^0, 8^1, 8^2, 8^3$ diye gider. Kullandığımız onluk sistem de bize bunu söyler, diğer sistemler de aynı mantıkla çalışır.

4. İkilikten Sekizlik ve On Altılık Sisteme Geçiş

Hoca tahtaya uzun bir ikilik sayı yazdı. Bu bir komut da olabilirdi, bir registerda saklanan değer de. Sayının on sekiz basamağı vardı:

```
010101110100110111
```

Bunu doğrudan onluk sisteme çevirmek zordur. $2^0, 2^1, 2^2, \ldots$ diye gidip 2^{16} gibi kuvvetleri hesaplamak gerekir. Hoca en yüksek basamak 0 olduğu için 2^{17}'yi atladı, 2^{16}'nın 65536 olduğunu hatırlattı. Uzun verilerde bu hesaplar yorucudur. Bunun yerine ikilikten sekizlik veya on altılık sisteme geçmek çok daha kolaydır.

4.1 Sekizlik Sisteme: Üçer Üçer Gruplama

Sekizlik sisteme geçerken bitleri üçer üçer gruplarız. Sebebi, $2^3 = 8$ olmasıdır: Her üç bitten sonra yeni bir sekizli başlar. 2^6 da 2^3'ün karesidir, yani sekizliler böyle devam eder. Üç bitle yazılabilecek en büyük değer 7'dir, bu da sekizlik sistemin en büyük rakamıdır.

Sayıyı öğrencilerle birlikte, en sağdan başlayarak grup grup çevirdik:

```
İkilik (3'erli):  010  101  110  100  110  111
Sekizlik:           2    5    6    4    6    7
```

En sağdaki grup $111 = 7$'dir. Sonuç $(256467)_8$ olur. Her grup kendi içinde birler, ikiler, dörtler basamağıyla okunur.

4.2 On Altılık Sisteme: Dörder Dörder Gruplama

Aynı mantıkla on altılık sistemde dörder dörder gruplarız. Dört bitle elde edilebilecek en büyük değer 15'tir. Bir sonraki basamağa geçersek 16'yı buluruz. Üçlü gruplamanın sebebi 7'ye kadar çıkabilmek, dörtlü gruplamanın sebebi 15'e kadar çıkabilmektir.

```
İkilik (4'erli):  0001  0101  1101  0011  0111
On altılık:          1     5     D     3     7
```

Soldaki grupta yalnız iki bit kalıyor (`01`). Bunun hiçbir önemi yoktur. Önüne 0 eklemek değeri değiştirmez: 0385 de sonuçta 385'tir.

Gruplar tek tek şöyle okunur:

- $0111 = 4 + 2 + 1 = 7$
- $0011 = 2 + 1 = 3$
- $1101 = 8 + 4 + 1 = 13$. On altılık sistemde A = 10, B = 11, C = 12, D = 13 olduğu için bu grup D'dir.
- $0101 = 4 + 1 = 5$
- $01 = 1$

Sonuç $(15D37)_{16}$ olur.

> **Hocanın vurgusu:** Gruplamaya her zaman en düşük anlamlı bitten (LSB), yani sağdan başlanır. Hoca, dördüncü sınıfa gelmiş olmasına rağmen en anlamlı bitten (MSB) başlayan öğrenciler gördüğünü söyledi.

Hoca bu sayıların söylenişini de karşılaştırdı: "1-5-D-3-7" ya da "2-5-6-4-6-7" gibi söylemek, on sekiz biti tek tek okumaktan çok daha kolaydır.

> **Ders sonrası not:** Derste sayının onluk karşılığı hesaplanmadı. Basamak değerleriyle hesaplanırsa $(15D37)_{16} = 89399$ çıkar.

5. Sekizlik ve On Altılıktan İkiliğe Geçiş

Ters yönde iki yol vardır.

5.1 Bölme Yöntemi

Çarpma yaptığımız yerde bu kez böleriz: Sayı sürekli 2'ye bölünür, kalanlar sondan başa doğru yazılır. Hocanın 7 ile başlayan ilk örneğinde bütün kalanlar 1 çıktığı için mantık tam görünmedi:

- $7 = 2 \cdot 3 + 1$, kalan 1
- $3 = 2 \cdot 1 + 1$, kalan 1
- Geriye 1 kaldı, artık bölemiyoruz.

Geriye doğru okuyunca 111 elde edilir. Hoca 15'i denemeyi düşündü, ama onda da bütün bitler 1 çıktığı için 13'e geçti:

- $13 = 2 \cdot 6 + 1$, kalan 1
- $6 = 2 \cdot 3 + 0$, kalan 0
- $3 = 2 \cdot 1 + 1$, kalan 1
- Geriye 1 kaldı.

Son bölümden başlayıp kalanları geriye doğru okuyunca $(13)_{10} = (1101)_2$ olur.

Soru: Bölme işlemlerinde sonuç nasıl okunuyordu?

Cevap: 13 örneğindeki gibi: Bölüyoruz, bölüyoruz, sonra geriden alıyoruz.

5.2 Basamak Değeri Yöntemi

Hocanın önerdiği yol budur. Örneğin D'yi ikiliğe açacağız. D, 13'e karşılık gelir. Dört basamağımız olacak ve bu basamakların değerleri 8, 4, 2, 1'dir.

1. 13'ün içinde 8 var mı? Var, o zaman en anlamlı basamak 1. Geriye 5 kaldı. Sayı 8'den küçük olsaydı, örneğin 7 olsaydı, bu basamak 0 olurdu.
2. 5'in içinde 4 var mı? Var, bu basamak 1. Geriye 1 kaldı.
3. 1'in içinde 2 var mı? Yok, bu basamak 0.
4. 1'in içinde 1 var mı? Var, bu basamak 1.

Sonuç 1101'dir.

> **Hocanın vurgusu:** Bilgisayar bölümü öğrencisi olarak 0 ile 15 arasındaki sayıların ikilik açılımları gözünüzün önüne gelmelidir. Bu ezber değildir, kullana kullana öğrenilir. 0000'dan 1111'e kadar olan karşılıklar bilinirse dönüşümler çok hızlı yapılır. Hoca uzun uzun bölmeyi tavsiye etmediğini söyledi.
>
> **Ders sonrası not:** Hoca 5.2'deki son adımda "geriye kalan 4'ün 1'i" dedi. Doğrusu, 4 de düşüldükten sonra geriye 1 kalır: $13 = 8 + 4 + 1$.

6. Kesirli Sayıların Dönüşümü

Virgüllü sayılar da bir sistemden diğerine geçirilebilir.

6.1 İkilikten Onluğa

Hocanın örneği $(1010.011)_2$ idi. Noktanın solundaki basamaklar $2^0, 2^1, 2^2, 2^3$ ile, sağındakiler $2^{-1}, 2^{-2}, 2^{-3}$ ile çarpılır:

$$1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3}$$

Solda $8 + 2 = 10$ kalır. Sağda $\frac{1}{4} + \frac{1}{8} = \frac{2}{8} + \frac{1}{8} = \frac{3}{8} = 0.375$ kalır. Sonuç 10.375'tir.

> **Hocanın vurgusu:** Sayı ikilik sistemdeyse basamaklarda 0 ve 1'den başka rakam olamaz. Bazı öğrenciler buraya 2 yazma hatası yapıyor. Bu olmaz.

6.2 Onluktan İkiliğe

Tersi biraz daha karışıktır. Kesirli kısım sürekli 2 ile çarpılır. Her çarpımın tam kısmı bir bit olur, kesirli kısımla devam edilir, kesirli kısım 0 olana kadar sürer. Hocanın örneği 0.6875:

```
0.6875 x 2 = 1.375  = 1 + 0.375   -> 1
0.375  x 2 = 0.75   = 0 + 0.75    -> 0
0.75   x 2 = 1.5    = 1 + 0.5     -> 1
0.5    x 2 = 1.0    = 1 + 0.0     -> 1   (kesirli kısım 0, dur)
```

Tam kısımlar yukarıdan aşağıya yazılır. Noktanın solu 0 olduğu için solda 0 kalır: $(0.6875)_{10} = (0.1011)_2$.

Sağlaması için bitleri $2^{-1}, 2^{-3}, 2^{-4}$ ile açarız:

$$\frac{1}{2} + \frac{1}{8} + \frac{1}{16} = \frac{8+2+1}{16} = \frac{11}{16} = 0.6875$$

Sayı 5 ile bittiği için 0'a kısa sürede ulaştık. 0.6873 olsaydı son basamakta 6 oluşacak ve dönüşüm uzayacaktı. O durumda belli bir yerde kesmemiz gerekirdi. Hoca bunun Sayısal Analiz dersindeki kesme hatalarından biri olduğunu söyledi.

7. İşaretli Sayılar ve İşaret Biti

Sayıyı ikiliğe çevirmeyi öğrendik. Peki bir RAM'de veya işlemcide pozitif ve negatif sayıları nasıl temsil edeceğiz? Sayının içindeki bir bitlik alanı işaret bilgisine ayırırız, kalanlar veriyi tutar. 8 bitlik bir alanda bir bit işaret (sign), yedi bit veridir (data).

Peki işaret biti nereye konmalı? Hoca canı isterse ortadaki bir biti işaret biti yapabileceğini söyleyip şöyle bir örnek verdi. Veri bitleri 1101011, ortadaki işaret biti de 1 olsun. O zaman sayı negatiftir, büyüklüğü de:

$$(1101011)_2 = 2^6 + 2^5 + 2^3 + 2^1 + 2^0 = 64 + 32 + 8 + 2 + 1 = 107$$

Yani sayı -107'dir. Ama bu iş hem bizi hem bilgisayarı yorar. Bilgisayar sayıyı çözerken aradaki dördüncü biti almak, işaretin solundaki ve sağındaki bitleri ayırıp birleştirmek zorunda kalır. Buna ekstra bir maskeleme gerekir.

Bu yüzden işaret biti ya en düşük ya en yüksek bite yazılabilir. Birçok sistemde en yüksek anlamlı bit (MSB) işaret biti seçilmiştir. Geri kalan bitler veri için ayrılır. Aynı örnekte en sola 1 yazılır, kalanına 1101011 gelir. Bilgisayar sayıyı çözerken artık yorulmaz.

> **Ders sonrası not:** Bu bölümde anlatılan, işaret biti ile büyüklüğü ayrı tutan gösterimdir. Bu derste biraz sonra kullanılacağı söylenen ikiye tümlemede aynı bitler farklı okunur: `11101011`, burada -107 olarak okunurken ikiye tümlemede -21 eder.

8. IEEE 754 Floating Point Gösterimi

8.1 Neden Bir Standart Gerekiyor?

Negatif ve pozitif sayıları, virgüllü sayıları ikiliğe çevirmeyi gördük. Peki herhangi bir reel sayı işlemcide, RAM'de veya bir komutta nasıl temsil edilir? Bunun standart olması gerekir, çünkü aynı sayı birçok şekilde yazılabilir. 0.6875'i 6.875×10^{-1} diye de yazabilirim. Bir başkası başka bir biçime dönüştürmeye kalkarsa 0 ve 1 karşılığının hassasiyeti farklı olur.

Hoca standart kavramını şöyle açıkladı: Herkes bir algoritma geliştirip bir problemi çözebilir. Ama herkesi ilgilendiren problemlerde belli bir standarda uymak gerekir. Bu standartları belli kurumlar koyar. IEEE'nin standartlarının da kimlik numarası gibi numaraları vardır. Kablosuz iletişim standardı IEEE 802.11 olarak kodlanmıştır. Reel sayıların gösterimi için kullanılan standart ise IEEE 754'tür. Konunun adı *Floating Point Representation on Binary System*, yani ikilik sistemde floating point sayıların gösterimidir.

8.2 Single ve Double Precision

IEEE 754 iki türde gelir: 32 bitlik türe single precision, 64 bitlik türe double precision denir. Precision, bir anlamda hassasiyet demektir.

Hoca kelimeyi açıklamak için COVID testi örneği verdi. 1000 kişiye test yapılsın.

- Gerçekte COVID olan 900 kişiden kaçını pozitif olarak işaretledik? Bu ölçüye **recall** denir.
- 900 kişiyi pozitif olarak işaretledik. Bunlardan kaçı gerçekten hasta? Bu ölçü **precision** olur.

Makine öğrenmesinde ve yapay zekada kullanılan precision bu mantıktadır. Single ve double precision ise verinin hassasiyetiyle ilgili bir kavramdır.

8.3 Üç Alan: İşaret, Exponent, Fraction

İşaretli tam sayılarda bit dizisini ikiye bölüyorduk, burada üçe böleriz: işaret (S), exponent ve fraction. Fraction, mantissa olarak da geçer. İşaret kolaydır: Artı için 0, eksi için 1. Exponent ve fraction'ın zor olmasının sebebi, sayıyı farklı şekillerde yazabilmemizdir. Standart bu alanların boyutlarını sabitlemiştir:

	İşaret	Exponent	Fraction	Toplam
Single precision	1 bit	8 bit	23 bit	32 bit
Double precision	1 bit	11 bit	52 bit	64 bit

8.4 Binary Normalization ve Değer Formülü

Karışıklığı gidermek için sayı *binary normalization* denen bir biçimde yazılır:

$$\pm 1.xxx\ldots \times 2^{y}$$

Sayının önünde işaret, noktanın solunda tek bir basamak, sağında 0 ve 1'ler, sonunda da ikinin bir kuvveti bulunur. Saklanan alanlardan sayının değeri şöyle bulunur:

$$X = (-1)^{S} \times (1 + \text{fraction}) \times 2^{\text{exponent}-\text{bias}}$$

Bias değeri exponent alanının bit sayısından gelir:

- Single precision: $2^{8-1} - 1 = 2^7 - 1 = 128 - 1 = 127$
- Double precision: $2^{11-1} - 1 = 2^{10} - 1 = 1024 - 1 = 1023$

Hoca bunu bir normalleştirme, belli bir aralığa çekme işlemi olarak tanımladı. Sayıyı saklarken üsse bias eklenir, saklanan bitlerden değeri bulurken çıkarılır.

> **Ek bilgi (derste anlatılmadı):** Bias büyük olasılıkla negatif üsleri de exponent alanında işaretsiz bir sayı olarak tutabilmek için vardır. Örneğin 2^{-1} gibi bir üs single precision'da $-1+127 = 126$ olarak saklanır. Böylece büyük olasılıkla exponent alanı için ayrı bir işaret bitine gerek kalmaz.

8.5 Örnek: 6.375'i Single Precision ile Saklamak

Verilen sayı 6.375. Üç alanı bulacağız.

1. İkiliğe çevirme. $6 = 110$. Kesirli kısım için 2 ile çarpma yöntemini kullanırız ve $0.375 = 0.011$ bulunur. Yani $6.375 = (110.011)_2$.

2. Normalize etme. Normalize etmek, noktanın solunda bir basamak kalması demektir. Noktayı iki basamak sola kaydırırız. Bunu dengelemek için 2^2 ile çarparız:

$$110.011 = 1.10011 \times 2^2$$

Hoca bunu onluk sistemle bağladı: 6.05'i 60.5 biçiminde yazınca 10^{-1} ile çarpıyorduk. Burada da nokta sola kaydığı için ikinin karesiyle çarpıyoruz.

3. İşaret. Sayı pozitif, yani $S = 0$.

4. Exponent. Üs 2'dir. Single precision'da bias 127 olduğu için $2 + 127 = 129$ saklanır. 129'u ikilik yazarız: $129 = 128 + 1 = (10000001)_2$. $2^7 = 128$ olduğu için 8 bit yeterlidir, dokuzuncu bite gerek yoktur.

5. Fraction. Noktanın sağındaki bitler 10011'dir. Bu bitlerin basamakları $2^{-1}, 2^{-2}, 2^{-3}, \ldots$ diye gider. Sağa doğru gittikçe hassasiyet artar. Single precision 23 bite kadar izin verir. İlk anlamlı kısımdan sonra kalan bitler 0 olur.

```
S   Exponent (8 bit)   Fraction (23 bit)
0   10000001           10011000000000000000000
```

Soru: Exponent kısmını anlayamadım.

Cevap: Normalizasyonda bulduğumuz sayıya IEEE 754 diyor ki single precision ise 127, double precision ise 1023 ekleyeceksin. Orijinalini bulmak istiyorsan da o değeri çıkaracaksın. Bu bir normalizasyon işlemidir, yani belli bir aralığa çekmedir.

Soru: Üsse mi ekliyoruz? Üs 2 olduğu için mi 2'ye ekledik?

Cevap: Evet, buradaki 2'ye ekliyoruz. Buradaki değer exponent'tir, bias da ona eklenir.

Geri dönüş. Saklanan bitlerden sayıyı bulmak için doğrudan formüle koyarız. Fraction'ı $2^{-1}, 2^{-2}, \ldots$ ile açarsak $\frac{1}{2} + \frac{1}{16} + \frac{1}{32} = 0.59375$ çıkar (ara adım eklendi). Exponent 129, bias 127'dir:

$$X = (-1)^0 \times (1 + 0.59375) \times 2^{129-127} = 1.59375 \times 4 = 6.375$$

Hoca aynı örneği double precision ile denememizi istedi.

8.6 Neden Tek Basamak? Avogadro Benzetmesi

Hoca bu kuralın bilimsel gösterimden geldiğini Avogadro sayısıyla anlattı. Avogadro sayısı 6.02×10^{23}'tür. Bu kadar tanecik bir mol eder. Mol aslında deste, düzine gibi bir ölçü birimidir. Bu sayıyı 60.2×10^{22} diye de yazabiliriz. Ama 60.2'yi açarken farklı bir şey çıkar, exponent ve fraction kısımları değişir. Belli bir standart olması için noktanın solunda tek basamak, sağında diğer basamaklar olacak şekilde düzenleme yapmak gerekir.

8.7 Soru: Noktanın Solundaki Basamak 0 Olsaydı?

Soru: Fraction'a noktanın solundaki 1'i almadık. O 1 yerine 0 olsaydı nasıl bir işlem yapacaktık? Sayı değişiyor, ama işlemlerde hangi kısım değişiyor?

Cevap: Hoca önce bunun kendi bildiği kadarıyla önemli olmadığını, exponent ve fraction'a noktanın solundaki kısımdan bir şey alınmadığını söyledi. Sonra 0.110011×2^3 yazımını önerdi: Fraction'a 110011 yazılır, exponent $3 + 127$ olur, bu da 6.375'i vermelidir. Ardından başka bir örneğe geçerken şunu ekledi: Burada fazladan bir 0 olmaz. 3'ü, 4'ü, 5'i, 9'u açın, en az bitle açtığınızda en soldaki basamak kesinlikle 1 olur. Örneğin 9, 1001'dir. Hiçbir zaman başta 0 kalmaz, o bir sabittir.

> **Ders sonrası not:** Hocanın ilk önerisi yanlış, son söylediği doğrudur. Formül noktanın solunda her zaman 1 varsayar, bu yüzden önerilen bitler 6.375 vermez. $3+127 = 130 = (10000010)_2$ ile $1.110011_2 \times 2^3 = 14.375$ çıkar. Hoca exponent'i tahtada `10000011` diye okudu; bu 131'dir ve $1.110011_2 \times 2^4 = 28.75$ verir. Doğru gösterim 8.5'teki gibidir: 1.10011×2^2.
>
> **Ders sonrası not:** Hocanın "hiçbir zaman 0 kalmaz" gerekçesi tam sayı kısmı olan sayılarda geçerlidir. 1'den küçük sayılar da negatif üsle aynı biçime gelir: $0.6875 = (0.1011)_2 = 1.011_2 \times 2^{-1}$. Baştaki basamağın 1 sayılmadığı istisna büyük olasılıkla exponent alanının tamamen 0 olduğu durumdur. Sıfır sayısı ve çok küçük (subnormal) sayılar büyük olasılıkla bu şekilde gösterilir.

8.8 23 Bite Sığmayan Sayılar

Sayı 0.375 değil de 0.373 gibi bir şey olsaydı ikilik açılımı uzun giderdi. 0.333... gibi sayıların ikilik açılımı da sonsuza kadar gider. Bu durumda fraction alanı 23 bite kadar doldurulur, geri kalan kısım hata olarak kalır.

8.9 Örnek: Bitlerden Sayıya

Hoca tahtaya şöyle bir bit dizisi yazıp değerini sordu:

```
S   Exponent (8 bit)   Fraction (23 bit)
1   10000001           01000000000000000000000
```

- İşaret 1, sayı negatif.
- Exponent 129, formülde $129 - 127 = 2$ olur.
- Fraction $0.01_2 = 2^{-2} = \frac{1}{4} = 0.25$.

Bazen formülde fraction'ı ikilik yazarlar, 0.01_2 gibi. Hoca bunun gerekmediğini, doğrudan 0.25 yazılabileceğini söyledi. Formüle koyarsak:

$$X = (-1)^1 \times (1 + 0.25) \times 2^{129-127} = -1 \times 1.25 \times 4 = -5.0$$

Hoca çalışma için 2.66 ve 3.66 gibi sayıları da önerdi. Burada tam kısımlar 3 için 1.1, 2 için 1.0 diye başlar. Bu tarz soruların hem sayıdan bitlere hem bitlerden sayıya çalışılmasını istedi.

9. Dört Bitlik Tablo: Sayı Sistemlerini Kısa Yoldan Görmek

Hoca, sayı sistemlerinin birbirine dönüşümünü daha mantıklı görmek için 4 bitlik tabloyu kurdu. 4 bit ikilik sistemde en fazla $2^4 = 16$ farklı durum yazılabilir. Tablo 0000'dan başlar, hepsi 1 olana kadar gider.

Tabloyu kurmanın kolay bir yolu vardır: Bitler basamak değeri kadar değişir. 2^0 sütunu birer birer, 2^1 sütunu ikişer ikişer, 2^2 sütunu dörder dörder, 2^3 sütunu sekizer sekizer değişir.

2^3	2^2	2^1	2^0	Onluk	On altılık
0	0	0	0	0	0
0	0	0	1	1	1
0	0	1	0	2	2
0	0	1	1	3	3
0	1	0	0	4	4
0	1	0	1	5	5
0	1	1	0	6	6
0	1	1	1	7	7
1	0	0	0	8	8
1	0	0	1	9	9
1	0	1	0	10	A
1	0	1	1	11	B
1	1	0	0	12	C
1	1	0	1	13	D
1	1	1	0	14	E
1	1	1	1	15	F

Tablodan tabanların kendisi de okunur: İlk 4 satır dörtlük tabanı, ilk 8 satır sekizlik tabanı, ilk 10 satır onluk tabanı, tamamı on altılık tabanı gösterir.

10. Bire Tümleme ve İkiye Tümleme

Elimizde +3 olsun, -3'ü bulmak istiyoruz. Bunun bir sürü gösterimi vardır. Bire tümleme ve ikiye tümleme (two's complement) denir, kullanılan ikiye tümlemedir.

10.1 İki Adımlı Yol

8 bitlik sayı sistemi kullandığımızı varsayalım:

```
+3                    : 0000 0011
Bire tümleyen         : 1111 1100   (0'lar 1, 1'ler 0)
Bire tümleyen + 1     : 1111 1101   -> -3
```

0'ları 1, 1'leri 0 yapınca elde ettiğimiz şey bire tümleyendir (one's complement). Buna 1 eklediğimizde ikiye tümleyeni buluruz.

10.2 Kısa Yol

Sağdan başlayıp ilk 1'i görene kadar, o 1 dahil, her şeyi aynen yazarız. Kalan bitleri ters çeviririz. +3 için en sağdaki 1 aynen kalır. Sonraki 1, 0 olur. Sonraki 0'lar 1 olur. Sonuç yine `11111101`'dir.

Aynı işlem ters yönde de çalışır. Elimizde -3 varsa +3'e gideriz: En sağdaki 1'i aynen yazarız, geri kalan 0'ları 1, 1'leri 0 yaparız. Sonuç yine `00000011` olur. Bit sayısı önemli değildir, 64 bit bile olabilir.

Soru: En yakın 1 derken neyi kastediyoruz?

Cevap: Örneğin elimizde 0100 var. Sağdan ilk 1'e kadar olan kısım (`100`) aynı kalır, geri kalanlar değişir. Sonuç `1100` olur (ara adım eklendi).

10.3 Kaç Sayı Yazabiliriz?

4 bitlik sayılarda pozitif ve negatifleri kodlayacaksak bir bit işarete gider. Bu durumda yazılabilecek aralık:

$$-2^3 \text{ ile } +2^3 - 1 \text{ arası, yani } -8 \text{ ile } +7 \text{ arası}$$

Hoca, bire tümlemenin bir sıkıntısı olduğunu söyleyip geçti: 0 iki defa yazılıyor, +0 ve -0 gibi.

11. Ara Konu: Programı Modüler Yazmak

Arada bir öğrencinin paylaştığı bir kod üzerine konuşuldu. Kod, $5x^9 - 3$ gibi bir fonksiyonu kullanıcıdan alıp girilen değer için fonksiyonu hesaplıyordu. Yani denklem programa girdi olarak string biçiminde veriliyor, program da function evaluation yapıyordu. Hoca bunu iyi bir başlangıç olarak değerlendirdi.

Eklediği nokta şuydu: Her şeyi `main` içine yazmak yerine fonksiyonlar kullanılmalı, program modüler hale getirilmelidir. Program modüler olunca işin içine pointerlar ve başka konular girecek. Bunlar bu derste anlatılacaktır.

Ayrıca bir terim sorusu netleşti: Tekrarlı yapının İngilizcesi *repetitive*'dir.

12. Algoritmanın Tanımına Dönüş

Dersin amacı yapısal programlamaya giriştir. Yapısal programlama yapabilmek için önce algoritma geliştirmek gerekir. Daha önce algoritmanın iki özelliğini söylemiştik: Komutlardan oluşur, bu komutlar belirli (unambiguous) ve sonlu (finite) sayıdadır. Bir de bir problemi çözer.

Bu problem illa bilgisayarda çözülecek bir problem değildir. Hayatın her alanındaki problemlere böyle bakılabilir. Hoca iki örnek verdi:

- Hastaneye giden biri COVID mi değil mi diye belli bir prosedürden geçirilir: Önce semptomlarına bakılır, sonra gerekli tetkikler yapılır.
- Askeriyede televizyonu açmanın, elektrik lambasının, radyonun çalıştırılmasının talimatları yazar. Bunlar da düpedüz algoritmadır.

12.1 Origami Örneği

Hoca ders kitabından bir slayt gösterdi. Amaç kağıttan bir kuş üretmektir, yani origami. Kitapta bir komut kümesi tasarlanmıştır: Bir sembol "kağıdı burada gösterildiği gibi katla" anlamına gelir, başka bir sembol başka bir işlem anlamına gelir. Bu semboller sınırlı ve belirlidir. Bunları toplama, çıkarma, çarpma, bölme veya karşılaştırma işlemleri gibi düşünebiliriz.

Bu komutlardan yola çıkarak adım adım işlem yapılır. Adımlar bizi bir sonuca götürür, sonsuza gitmeyiz. Örnek, algoritmanın tanımını görsel olarak sunar.

12.2 Algoritma Nasıl İfade Edilir?

Bir algoritma şu yollarla ifade edilebilir:

- Program olarak yazılır.
- Sözlü anlatımla (oral presentation) ifade edilir.
- Akış diyagramı (flow chart) yapılır.
- Sözde kodla (pseudocode) yazılır.

13. Üç Temel Yapı

Hoca geçen dersin sonundaki kek tarifini hatırlattı. Tarifte ne kadar un dökületeceği, yumurtayla şekerin çırpılması gibi adımlar vardı. Ne kadar çırpılacağı sorulunca "köpük olana kadar" cevabı verilmişti. Tekrar sayısı belli değildir, bir koşula bağlıdır.

Algoritmalar üç yapıdan oluşur:

- **Ardışıl (sequential) yapılar:** Toplama, çıkarma, atama, == gibi temel mantıksal ve matematiksel işlemler. Yazdırma da burada düşünülebilir.
- **Şartlı (conditional) yapılar:** `if`.
- **Tekrarlı (repetitive) yapılar:** İkiye ayrılır.
  - Belirli tekrar (definite): Ne zaman biteceği bellidir. `for` grubu.
  - Belirsiz tekrar (indefinite): Ne zaman biteceği belli değildir. `while` grubu. `do-while` da buradadır. Bazı dillerde bu yapı `repeat-until` olarak geçer.

  Hocanın vurgusu: Algoritma geliştirirken dördüncü bir yapı yoktur.

13.1 Kekik ve Nane

Şartlı yapı için hocanın örneği yemek tarifinden geldi:

- "Evde kekik yoksa yerine nane koy" hem `if` hem `else` içerir.
- "Evde kekik yoksa hiçbir şey koyma, kekiksiz olsun" yalnız `if` içerir, `else` yoktur.

Keki pişirip servis yaparken tabaklara aktarmak da bir atamadır. = operatörü gibi, bazı dillerdeki += ve -= gibi. Bunların hepsi temel işlemlere gelir.

13.2 == Neden Şartlı Yapı Değildir?

> **Hocanın vurgusu:** Değer üreten, sonuç üreten her şey sequential bir yapıdır. == bir operatördür, conditional değildir. Şartlı yapıyı sağlayan, sonuca göre farklılaşmayı sağlayan `if`'tir.

Soru: Eşitliğin iki tarafı eşitse == bir değer üretmiyor mu? Neden sequential?

Cevap: Evet, iki taraf eşitse 1 veya true üretir. Değer ürettiği için sequential'dır. Sonucuna göre bir taraf çalışacak, öbür taraf çalışmayacak demiyoruz. Yalnız bir değer üretiyor, sonra sıradaki işleme geçiliyor.

Şu satıra bakalım:

```
a = b == c;
```

Bilgisayar tarafında önce `b == c` yapılır. Bu genellikle işlemcide bir compare komutudur. Karşılaştırmadan 1 veya 0, yani true veya false, bir değer gelir. İkinci adımda bu değer `a`'ya atanır. Yani burada iki tane sequential işlem vardır.

`if`'te ise durum farklıdır:

```
if (b == c) {
    /* parantezin içi 1 ise burası çalışır */
} else {
    /* değilse burası çalışır */
}
```

14. Yol Tarifi Örneği

Hoca bir öğrenciden, İstanbul'a yeni gelmiş bir turiste Ataköy'den Esenler Otogarı'na yol tarif etmesini istedi. Turist bir dili biliyor, tarif eden de o dili biliyor, ama turist başka hiçbir şey bilmiyor, bir çocuk gibi. Otobüse bile nasıl binildiğini bilmiyor. Yani her şey tarif edilmeli. Tarif, soru-cevaplarla şöyle gelişti:

1. Marmaray'ın girişine git.
2. Otomata para yerleştir, düğmeye bas, kartı al.
3. Turnikelere git, kartı turnikeye yerleştir, turnikeden geç, kartını al.
4. Durakta bekle, Marmaray gelince bin.
5. Şirinevler durağına kadar seyahat et, orada in.
6. Şirinevler'de yine otomata git, para yerleştir, düğmeye bas, bileti al.
7. Turnikeden geç, metroyu bekle, metro gelince bin.
8. Otogar durağı olana kadar seyahat etmeye devam et, in.

Hoca arada iki önemli soru sordu.

"Adamın parası yoksa?" Öğrenci önce şaka yaptı, ama hoca burada bir şart olduğunu gösterdi: Parası yoksa ATM'den para çekecek. O zaman bu da tarif edilmeli: ATM'ye kartı yerleştir, şifreyi gir, para çek, kartı al, parayı al, makbuzu al. Yani şartlı yapı ve ona bağlı alt adımlar.

"Şirinevler durağına kadar seyahat et" hangi yapıdır? Öğrenci önce "if Şirinevler in" dedi. Ama burada bir süreç vardır, yapılacak iş seyahat etmektir. Hoca farklı ifade biçimlerini şöyle sıraladı:

- "Durak Şirinevler değilken seyahate devam et" bir `while` gibidir. Durak sayısını tam bilmediğimiz zaman böyle deriz.
- "Beş durak sonra in" bir `for` gibidir. İstanbul'u çok iyi biliyor, durak sayısından eminsek böyle deriz. Hoca "üç durak sonra" diye de örnekledi.
- Seyahat en az bir durak süreceği için bu `do-while` ile de ifade edilebilir.

15. Barcelona Metrosundaki Turnike

Hoca, Barcelona metrosundaki bir turnikenin videosunu gösterdi. Bizdeki turnikelerde kart okutulunca turnike doğrudan açılır. Oradaki sistem ise farklıdır. Kart makinenin içine takılır, bir taraftan girip öbür taraftan çıkar. Turnike, kart oradan alınmadan açılmaz. Bir sensör kartın alındığını anlar. Geçer geçmez de turnike kapanır.

Hoca buradaki durumu şu adımlarla anlattı:

1. Kart takılana kadar makine çalışmaz, bekler.
2. Kart takılır ve öbür taraftan çıkar. Kart alınana kadar turnike açılmaz, bekler.
3. Kart alınınca turnike açılır. Geçiş olana kadar bekler. Belli bir süre geçilmezse uyarı vermeye başlar.
4. Geçiş olunca kapanır.

Bu tasarımın bir faydasını da öğrenciler söyledi: Kartı unutup gidemezsiniz, çünkü geçebilmek için onu teslim almanız gerekir.

Hoca bu turnikeyi Ayrık Matematik dersinde bir sonlu durum makinesi (finite state machine) sorusu olarak sormuştu. Burada önemli olan şudur: Bütün adımlar yine üç yapıdan birinden oluşur, dördüncü bir şey yoktur. "Bir yeri görene kadar git" bir `while`, "üç durak sonra in" bir `for` gibidir.

16. Robota Araba Çizdirme

Fatih Hoca'nın ev çizdirme etkinliğinin benzeri olarak hoca öğrencilerden bir robota araba çizdirmelerini istedi. Robot insana benzer bir robottur. "Çizgi çiz", "sağa doğru git", "dur", "kalk" gibi komutları bilir. Hoca lisede Pascal'da `draw line` fonksiyonuyla başlangıç koordinatını ve bitişi vererek çizgi çizdiklerini hatırlattı. Burada öyle yapılmayacak, robotla konuşulacak.

Etkinlikte ortaya çıkan noktalar şunlardı:

- Bir öğrenci “bir dikdörtgen çizsin” dedi. Hoca: Dikdörtgen çizmeyi bilmez, çizgi çizebilir. Bu yüzden önce “kalemi al” denir.
- “Çizgi çekmeye başla”, “dur”, “sağa doğru çizmeye başla”, “dur”, “yukarı doğru çizmeye başla” gibi komutlarla gövde kuruldu. Uzunluk birimi olmadığı için “dur” komutu devreye girdi.
- “Başladığın noktaya kadar çiz” denildi. Hoca: Başladığı noktayı bilmez, sağa sola bir komut verilmeli. “Sola doğru çiz” denildi.
- Çizim hatalı olunca “silgiyi al” komutu denendi.
- Tekerlekler için “yuvarlak çiz” komutunun tanımlı olduğu kabul edildi ve “köşeyi merkez kabul ederek bir yuvarlak çiz” denildi.
- İkinci öğrenci, arabanın sağına eklediği kısım için “5 birim sağa çiz” ve “45 derece açıyla köşeyle birleştir” gibi komutlar kullandı. AutoCAD’in de böyle çalıştığı söylendi.
- Üçüncü öğrenci noktalara ad vermeyi önerdi: 1, 2, 3, …, 9. “8. noktadan aşağı çizgi çek” demek tarifi çok kolaylaştırdı. Hoca bunu nesne yönelimli düşünmeye benzetti.

  Hocanın vurgusu: Robota söylenen her operasyon tanımlı olmalıdır. Çizginin nasıl çekildiğini bilmesi gerekir, burada bildiğini varsaydık. “Sağa beş birim” derken bir döngü vardır. Bu tanımlamalar yapılmazsa algoritma olmaz, muğlak (ambiguous) şeyler ortaya çıkar.

17. Tanımlı Olmayan İşlem Kullanılamaz

Hoca aynı noktayı bir bilgisayar problemiyle gösterdi. 10 ile 80’in en büyük ortak bölenini (GCD) hesaplayacağız. Birisi diyor ki önce iki sayının minimumu gerekiyor ve `min(10, 80)` yazıyor. Bu bir fonksiyondur, algoritma değildir. Minimumun nasıl bulunacağını tarif etmek gerekir.

Sebep şudur: İşlemcinin en temel seviyesinde minimum bulan bir komut yoksa bu çalışmaz. Programlama dili seviyesinde iki parametre alıp minimumu döndüren bir şey varsa kullanılabilir. Ama burada algoritmik bakış açısıyla baktığımız için minimumu nasıl bulacağımızı tarif ederiz.

18. İyi Algoritmanın Ölçütleri

Dersin amacı yalnız algoritma geliştirmek değil, daha iyi algoritma geliştirmektir. Peki iyi ya da kaliteli algoritma ne demektir? Ölçütler öğrencilerle birlikte çıkarıldı.

- **Planlı olması:** Bir öğrenci, önceden düşünülmüş adımlardan oluşan algoritmayı söyledi. Hoca bunu yaklaşım (approach) kavramına bağladı: Bir problemi çözerken önce kafanızda bir çözüm tasarlarsınız, bu yaklaşımdır. Bunu belli adımlara dökersiniz, bu da algoritmadır. Algoritma gelişigüzel yazılmaz.
- **Doğruluk (correctness, accuracy):** İstediğimiz sonucu vermesi. Burada çizgi çizilmesi bekleniyorsa çizgi çizmesi.
- **Kaynakların verimli kullanımı:** Döngülerin gereksiz yere fazladan dönmemesi gibi. Kaynaktan kasıt zaman (time) ve bellek (memory), bir de güç (power). Hocaya göre asıl üzerinde durduğumuz kaynak zaman ve bellektir.
- **Optimum sonuçlar:** Kesin (exact) sonuca ulaşamadığımız durumlarda optimum sonuçları kabul edebiliriz.
- **Netlik:** Yönergelerin açık ve anlaşılabilir olması.
- **Kullanıcı dostu olması (user friendly):** Bir öğrenci okuyanın anlayabilmesini ve görünüşü söyledi. Hoca bunu insan-bilgisayar etkileşimi (human computer interaction) konusuna bağladı. Bu daha çok algoritmayla değil, grafik arayüzle, kullanıcıya bakan front end tarafıyla ilgilidir.

Bunların hepsinden önce bilgiye sahip olmak gerekir. Bilmeden bir iş yapılamaz. Bir şeyin kaliteli olduğunu anlamak için yapısını anlamak, analiz etmek, açıklarını bulmak gerekir.

18.1 Ölçütlerin Örnekleri

Doğruluk. Yarın havanın kaç derece olacağı bir algoritmayla, örneğin yapay zekayla bulunabilir. Biri %98 doğrulukla bulduğunu söylerken diğeri %99 doğrulukla bulduğunu söyler. Bu

önemli bir metriktir. Bir oyun motorundaki operasyonlar da, bir öneri sistemi de doğru sonuç vermelidir. Karar destek sistemleri (Decision Support Systems) bu tür sistemlerin genel adıdır. Hoca uç bir örnek de verdi: Sesimden babamın vergi kimlik numarasını bulmak. Algoritma önce sesi analiz edip beni, benden babamı, ondan da numarayı bulmalıdır.

Kaynak kullanımı. Hocanın benzetmesi marketti: Markete girip kasadan hızlı çıkmak gibi, algoritma da işlemciden hızlı tamamlanıp çıkmalıdır. Bütün sıralama algoritmaları bu konuda birbiriyle rekabet halindedir. Merge Sort "daha hızlıyım ama daha fazla bellek kullanırım" der. Quick Sort "hem az bellek kullanırım hem hızlı sonuç üretirim" der.

Soru: Optimizasyona pi sayısını örnek verebilir miyiz?

Cevap: Verebiliriz. Pi sayısını üreten algoritmalar var. Hoca gelecek haftalarda Monte Carlo algoritmasını kodlamayı düşündüğünü söyledi. Ne kadar hassas yaklaşırsak o kadar kesin sonuçlar elde ederiz.

> **Ek bilgi (derste anlatılmadı):** Pi için kullanılan Monte Carlo yöntemi büyük olasılıkla rastgele noktalar üretip bunların ne kadarının bir çemberin içine düştüğünü saymaya dayanır. Nokta sayısı arttıkça sonuç büyük olasılıkla gerçek değere yaklaşır.

Soru: En az kodla en doğru sonucun üretilmesi de bir ölçüt olabilir mi?

Cevap: Koddaki metin açısından bakıyorsak bunu her zaman doğru kabul etmek mümkün değildir. Küçük bir kod parçasının arkasında bir sürü makine kodu olabilir. MATLAB'de bir satırla yazdığınız şeyi belki C'de 300 satırla yazarsınız. Bu tam ölçülebilir bir şey olmaz. Makine kodu kastediliyorsa doğru olabilir.

19. Ödünleşim (Trade-off)

Bu metriklerin hepsi aynı anda en iyi olamaz. Zaman zaman birinden ödün veririz. Birkaçı bizim için çok önemli, diğerleri daha az önemli olur. Buna ödünleşim (trade-off) denir.

- **Hız öne çıkar:** Sizin için hız önemlidir, bellek istediği kadar kullanılsın.
- **Bellek öne çıkar:** Gömülü sistemde veya robotikte bellek yetersizdir, verimsiz kullanılamaz. Belleği zorlamak bir süre sonra hızı da etkiler veya başka işlemleri engeller.
- **Süre ikinci planda kalır:** Toplu işlem (batch) verirsiniz. Örneğin bir milyon kişiye mail gönderilecek. Bunu sisteme girdi olarak verirsiniz, ne zaman biterse biter.

  Hocanın vurgusu: Bu ödünleşim tamamen probleme yönelik bir yaklaşımla (problem-oriented approach) belirlenir. Hangi ölçütün öne çıkacağını problem söyler.

20. İyi Algoritma Nasıl Gelişir?

İyi algoritma ilk seferde olmaz. "Ben yaptım, en iyisi oldu" diye bir şey olsaydı herkes pes edip giderdi. Oysa rekabet vardır. Hoca, bilimi tanımlarken çok sevdiği sözü aktardı: *"Science is approximation to the truth."* Yani bilim gerçeğe yaklaşmaktır. Birisi benim hatamı görüp düzeltir, ben bir başkasının hatasını düzeltirim. Algoritma da böyledir. Süreç, tecrübeye dayalı iteratif bir süreçtir.

İnternet tarayıcıları buna örnektir. Kimi belleği iyi kullanır, kimi grafik arayüzüyle öne çıkar. Hoca temsili oranlar vererek herkesin tek bir tarayıcıya yönelmediğini söyledi. Telefon markalarında da böyledir: Herkesin bütçesi, hız beklentisi gibi kendi öncelikleri vardır. Bu tercihlere hitap edebilmek gerekir.

Soru: Algoritmamızın verimli olup olmadığını nasıl tespit edebiliriz?

Cevap: Bu ders ve sonraki dersler (gelecek dönemin yapısal programlama dersi, veri yapıları, algoritma analizi) hep bunun üzerinedir. Problemler kendi içinde sınıflandırılır. Problem sınıflarına uygun en kötü ve en iyi durum analizleri vardır, onlar anlatılacak. Kendi algoritmanızın daha iyi olduğunu iddia etmek için başkalarının ne yaptığını analiz etmek gerekir. Selection Sort'un, Bubble Sort'un ve kendi algoritmanızın metriklerine bakarsınız. Zaman veya bellek açısından hangilerinden daha iyiyse "bu algoritma ona göre daha iyidir" diyebilirsiniz.

20.1 Bir Buçuk Döngü

Hoca birinci sınıf öğrencilerine "bir döngüyle sıralama yapmaya kalkmayın, en az bir buçuk döngüyle yapabilirsiniz" der. Bir döngüyle gerçekten yapabilen çığır açmış olur. Hoca counting sort gibi algoritmaların bu işi bir döngüyle yaptığını, ama normal bir sıralama algoritmasından söz ettiğini belirtti.

Soru: Bir buçuk döngü nasıl oluyor?

Cevap: While içinde for gibi, özyinelemeyle sağlanıyor. Bir buçuktan kastım $n \log n$ karmaşıklıktır. Daha sonra bakacağız.

> **Ders sonrası not:** Hoca bir döngüyle sıralamanın aksinin "şu ana kadar ispat edilmediğini" söyledi. Bu durum büyük olasılıkla karşılaştırmaya dayalı sıralama için daha kesindir: Bu sınıfta en kötü durumda $n \log n$ alt sınırı büyük olasılıkla kanıtlanmıştır. Hocanın ayırdığı counting sort karşılaştırmaya dayanmaz.

20.2 Tecrübe ve Bilgi: Hıçkırık Hikayesi

Hoca, bilgi ile tecrübenin birleşmesini kendi başından geçen bir olayla anlattı. Bir yolculukta saatlerce geçmeyen bir hıçkırığa tutulmuş, hastanede yapılanlar ve çevresindekilerin önerileri işe yaramamış. Bir hekim ona bir bardak suyu tek nefeste içmesini söylemiş, sorun hemen çözülmüş. Hocaya göre o hekimin bilgisi ile tecrübesi orada birleşmişti. Birçok meslekte olduğu gibi algoritma geliştirmede de iyi sonuç ikisi bir araya gelince ortaya çıkar.

20.3 Algoritma Geliştirmenin Algoritması Yok

Peki iyi algoritmayı nasıl öğreneceğiz? Bunun için bir ders veya hazır bir plan yoktur. Arkadaşlarımız ve kitaplar yardımcı olabilir. Ama algoritma geliştirmek, algoritma geliştirerek öğrenilir. Hoca bunu Fatih Hoca'nın bir sözüyle destekledi: Algoritma geliştirmenin bir algoritması yoktur.

Öğrenme, hataların en aza indirilmesidir. Bildiğiniz şeyleri tekrar ederek pekiştirirsiniz, pekiştirmediğiniz şey geçici bellekte kalır. Banu Hoca da birinci sınıfta yaşadığı benzer zorluğu hep anlatır. "Herkes yapıyor, ben niye yapamıyorum?" diyen kişi çoğu zaman yeterince hata yapmamış, hatalarını görmemiştir. Bildiğini zanneder, ezberlediğini fark etmez, yeni bir şey gelince afallar. Bu normaldir. Hoca kendisi için de çok zor bir algoritmanın aynı durumu yaratabileceğini söyledi. Kendi tecrübesinin de meslek lisesinden başlayıp Pascal, C, Basic, Visual Basic, sonra veri yapıları, algoritmalar ve yapay zeka ile devam ettiğini anlattı. Bu süreç devam eder, "ben oldum" diye bir nokta yoktur.

21. 1'den N'e Kadar Toplam

Hoca, algoritmanın iyileştirilebileceğini basit bir problemle gösterdi: $1 + 2 + \cdots + N$ toplamını bulan program.

21.1 Döngüyle Çözüm

Başlangıcı, bitişi ve artışı belli olduğu için adım sayısı bellidir, `for` kullanılabilir. C'de değişkenin tipi de söylenir: Hem tip belirtilir, hem tanım yapılır, hem ilk değer atanır. Hocanın derste $N = 10$ için sorduğu duruma göre tam program şöyledir:

```c
#include <stdio.h>

int main(void)
{
    int n = 10;
    int i = 0;
    int toplam = 0;

    for (i = 1; i <= n; i++)
        toplam = toplam + i;
```

```
    printf("toplam = %d, dongu sonu i = %d\n", toplam, i);
    return 0;
}
```

Program `gcc -Wall` ile uyarısız derlenip çalıştırıldı. Çıktısı `toplam = 55, dongu sonu i = 11` oldu.

Hoca işlemleri bir birimlik operasyon sayarak tek tek topladı:

İşlem	Kaç kez
`i = 0`	1
`toplam = 0`	1
`i = 1`	1
`i <= n`	N
`toplam + i`	N
atama	N
`i++`	N

Toplam $4N + 3$ eder. $N = 100$ girilirse 403 işlem yapılır. Hoca "atladığım bir şey olabilir" diye de ekledi.

Soru: $N = 10$ girildiğinde döngüden sonra `i`'nin değeri kaçtır?

Cevap: 11'dir. `for` döngüsünün çalışma mantığı şöyledir:

1. `i = 1` bir defa yapılır.
2. `i <= n` kontrol edilir.
3. Doğruysa gövde çalışır: `toplam = toplam + i`.
4. `i++` yapılır.
5. 2. adıma dönülür. Kontrol yanlış çıkınca döngüden çıkılır.

Ders sonrası not: Sonucun 11 çıkması, `i <= n` kontrolünün N değil $N + 1$ kez yapıldığını gösterir, çünkü döngüden çıkmak için son bir kontrol gerekir. Kontrol sayısını da bir sayaçla ölçtük: $N = 100$ için 101 kez yapılıyor. Bu sayılırsa toplam $4N + 4$, yani $N = 100$ için 404 olur. Bu fark sonucu değiştirmez: İşlem sayısı N ile doğrusal büyür.

Hoca küçük bir iyileştirme yaptı. İlk değer atamasını tanımlamaya taşıyıp `for`'un o kısmını boş bırakırsak bir işlem azalır, $4N + 2$ olur:

```
int i = 1;
int toplam = 0;

for (; i <= n; i++)
    toplam = toplam + i;
```

Bu parça da tam programın içinde `gcc -Wall` ile uyarısız derlendi ve aynı sonucu verdi.

21.2 Gauss Formülü

Matematikten bildiğimiz bir bilgiyi bilgisayar bilimiyle birleştirirsek daha iyisini yaparız. Hocanın anlattığı Gauss mantığı şudur: 1 ile N toplanırsa $N+1$, 2 ile $N-1$ toplanırsa yine $N+1$ eder. Bütün çiftler aynı toplamı verir.

$$\sum_{i=1}^{N} i = \frac{N(N+1)}{2}$$

```
#include <stdio.h>

int main(void)
{
```

```c
    int n = 10;
    int tp;

    tp = n * (n + 1) / 2;

    printf("toplam = %d\n", tp);
    return 0;
}
```

Bu program da `gcc -Wall` ile uyarısız derlendi. Çıktısı `toplam = 55` oldu.

İşlemleri sayalım: bir toplama, bir çarpma, bir bölme, bir atama. Toplam 4 birim. N ister 100, ister 1000, ister bir milyon olsun, işlem sayısı 4'tür. Döngülü çözümde ise 100 için 402, 1000 için 4002, yüz bin için 400002 işlem yapılır.

Bellek açısından da kazanç vardır. Döngülü çözümde `i`, `tp` ve `n` olmak üzere üç eleman, formülde `tp` ve `n` olmak üzere iki eleman kullanılır. `i`'ye gerek kalmaz.

Döngülü çözüm de bir algoritmadır: Belirlidir, sonludur, kuralları sağlar. Ama ikinci çözüm hem zaman hem bellek açısından daha iyidir.

> **Ders sonrası not:** Hoca N için bir milyon örneğini işlem sayısı açısından verdi. Tam kodda buna dikkat etmek gerekir: Bir milyon için toplam 5×10^{11} civarındadır. Bu değer büyük olasılıkla 32 bitlik `int`'e sığmaz. Yani bu büyüklükte iki kodda da daha geniş bir tip gerekir.

22. Problemi Genişletmek: m'den n'e Kadar Toplam

Hoca problemi biraz değiştirdi: 1'den N'e kadar değil de m'den n'e kadar toplayalım, n'nin m'den büyük olduğunu biliyoruz. Yine terim toplamı formülü kullanılır: Terim sayısı, son terim eksi ilk terim bölü artış miktarı artı birdir. Bu, son terim ile ilk terimin toplamının yarısıyla çarpılır. Hoca 2'den 8'e kadar olan sayıları örnek gösterdi:

$$\text{Terim sayısı} = \frac{8-2}{1} + 1 = 7, \quad \text{Toplam} = 7 \cdot \frac{8+2}{2} = 35 \text{ (ara adım eklendi)}$$

Sonra özel bir durum sordu: Aralık $-n$ ile $+n$ arasındaysa formülü işletmeye gerek yoktur. İkişer ikişer toplanan terimler birbirini götürür, sonuç 0'dır. Yani özel durumları görürsek daha da az işlem yaparız. Böyle algoritmalarda süre girdi boyutuna bağlı değildir.

> **Ders sonrası not:** Hoca, m ve n'den biri pozitif, biri negatif olduğunda "doğrudan $m-n$ yaparım" dedi. Bu genel olarak doğru değildir. Hocanın kendi örneğinde $m = -n$ için bu ifade $-2n$ eder, oysa toplam 0'dır. Doğrusu, formülü genel hali ile kullanmaktır: $\sum_{k=m}^{n} k = \frac{(n-m+1)(m+n)}{2}$. Burada $m = -n$ konursa da sonuç 0 çıkar.

23. Çözülmeyi Bekleyen Problemler

Bunun gibi birçok problem vardır ve onlar için algoritmalar geliştirilir.

- **PageRank:** Google'ın web sayfalarının benzerliğini ve güvenilirliğini birleştiren arama yöntemi. Hocaya göre bu çok önceden elle yapılıyordu, otomatize edildiği için Google bir dev haline geldi.
- **Yüz tanıma:** Hâlâ tam anlamıyla çözülmüş değildir, hatalar olabilir.
- **Görüntü ve ses aktarımı:** Görüntülü görüşmede bir uygulamada görüntü daha az takılıyorsa orada daha iyi bir algoritma geliştirilmiştir.

Bir algoritma diğerlerinin hatasını azaltıyorsa onları **outperform** eder, yani geride bırakır. Hoca aynı anlamda *surpass*, *eclipse* ve *excel* kelimelerini de saydı. Problemlerin hepsi tam çözülmüş değildir, yenileri de ortaya çıkar. Hocaya göre bu, mesleğimiz için önemli bir avantajdır.

Algoritma eksiği olanlar için hoca code.org'u önerdi. Sitenin çocuklar için geliştirildiğini, ama arayüzünün ve uygulamalarının çok iyi olduğunu söyledi. Fatih Hoca'nın sayfasındaki ders 1

ile ilgili üç soruyu da gelecek derste konuşacaklarını, örneğin bir ödünleşim örneği belirlemeyi veya çözülmeyi bekleyen problemleri (*unsolved problem* diye) araştırmayı önerdi.

> **Hocanın vurgusu:** "C'yi anlayamadım" diyen birine hoca şunu sorar: Sorunun C'yi veya Java'yı bilmek mi, algoritma geliştirmek mi? Çoğu kez cevap şudur: "For'u, while'ı biliyorum, ama bir problem gelince nasıl uyarlayacağımı bilemiyorum." Bu, yeterli algoritma tecrübesi olmadığının semptomudur. Tedavisi sürekli problem çözmektir.

24. Şifreler, Kaba Kuvvet ve Hash Fonksiyonları

24.1 Olası Şifre Sayısı

Hoca çözülmeyi bekleyen problemlere şifre çözmeyi örnek verdi ve bunu dersin başındaki ikilik sisteme bağladı. 4 bitlik kodlamada $2^4 = 16$ farklı durum vardı, çünkü her basamakta iki seçenek vardı. 500 farklı karakterimiz olsa ve 10 haneli bir şifre oluştursak, karakterler tekrar edebildiği için her hanede 500 seçenek olur:

$$500^{10} \text{ olası şifre}$$

Bu şifreyi kaba kuvvetle (brute force) çözmek için döngünün bu kadar dönmesi gerekir. Sürenin ölçülmesi, yani karmaşıklık analizi de bir problemdir. Hoca, 500 karakterlik bir kümeden 10 veya 15 karakterlik rastgele şifre üreten bir programı da permütasyon problemi örneği olarak verdi.

> **Ders sonrası not:** Derste sayı hesaplanmadı: $500^{10} \approx 9.77 \times 10^{26}$'dır.

24.2 Kaba Kuvvet Saldırılarına Karşı

Facebook veya Gmail şifre girerken 3 veya 4 deneme hakkı tanır. Bu bir engeldir. Böyle bir sınır olmasaydı bütün olasılıklar denenip şifre çözülebilirdi. CAPTCHA da otomatik denemeyi engeller.

24.3 Hash Fonksiyonları

Web sayfaları şifrenizi açık biçimde tutmamalıdır. Şifre geri döndürülemez bir fonksiyondan, özetleme ya da hashing fonksiyonundan geçirilir. Veri tabanında bu özet saklanır. Giriş yaptığınızda girdiğiniz şifre tekrar hash'lenir ve saklanan değerle karşılaştırılır. Aynı girdi aynı özeti verdiği için doğru girilip girilmediği buradan anlaşılır.

Bu yüzden şifrenizi unuttuğunuzda site size yeni şifre oluşturmanızı söyler: Eskisini geri çözmek mümkün değildir.

> **Hocanın vurgusu:** Bir site "şifrenizi mail ile gönderdik" diyorsa, şifreniz güvenli saklanmıyor demektir. Veri tabanında kullanıcı adının karşısında 123456 gibi açıkça duruyordur.

MD5 gibi bazı hash fonksiyonları kaba kuvvetle çözülebilmektedir. Hoca 512'lik olanlar gibi daha üst seviye fonksiyonların henüz çözülmediğini söyledi. Ancak "şu an çözülemez" dediğimiz bir problem birkaç yıl sonra çözülebilir.

Soru: Şifreyi neden geriye dönüp bulamıyor?

Cevap: Çok fazla farklı durum var, hepsini tek tek deneyip çözme imkanı yok. 500 yerine 50 bin karakterlik bir küme olsa çok daha zorlaşır. Hash sonuçta bir fonksiyondur: x girdisi y üretir, y'den x'i bulmak için çok fazla iterasyon gerekir. Normal bir sürede çözülemez.

Soru: Veriyi çevirirken veri kaybı oluyor mu?

Cevap: Bazı durumlarda oluyor, ama asıl neden bu değil. Asıl neden problemin çözülemez büyüklükte (intractable) olmasıdır: Bir şeyi 50 bin üzeri bin defa döndürmek üstel (exponential) zamana girer, polinom zamandan çıkar.

> **Ders sonrası not:** Hoca 50 bin karakterlik örnekte önce 10 haneli şifreden, sonra "50 bin üzeri bin"den söz etti. 10 haneli şifrede üs 10'dur. "Bin" büyük olasılıkla yalnız büyüklüğü göstermek için söylendi.

Eliptik eğrilerle ilgili bir soruya hoca, kişisel verilerin güvenliği protokollerinde verinin hash'lenmiş tutulduğunu anlatarak cevap verdi. Enigma sorusuna da onun da bir kodu olduğunu ve belli bir şekilde çözülmesi gerektiğini söyledi. Hocaya göre kriptoloji uzun bir konudur, burada girilmedi.

> **Ders sonrası not:** Hoca "bunu tersinden yapmak da düzünden yapmak da zor" dedi. Hash fonksiyonlarında düz yön büyük olasılıkla hızlıdır, zor olan ters yöndür. Hocanın anlattığı giriş kontrolü de bunu gösterir: Her girişte şifre yeniden hash'lenir. Ayrıca hoca isim ve soy ismin de hash'lenerek tutulduğunu söyledi. Geri okunması gereken veriler büyük olasılıkla hash'lenmez, geri çözülebilir biçimde şifrelenir. Hash ile şifrelemeyi aynı kelimeyle anmak da büyük olasılıkla bu yüzden yanıltıcıdır.

24.4 Polinom Zaman ve Problem Sınıfları

Problemleri sınıflandırırken polinom tipli sınıflar ve non-deterministik polinom tipli sınıflar diyeceğiz. Bir problemi iç içe iki döngüyle n^2, üç döngüyle n^3 defa çözüyorsam bu polinom tiplidir. Polinom tipli olmayan problemlerde çözüm polinom zamanda yapılamamıştır, yapılıp yapılmayacağı da henüz belli değildir. Belki on yıl sonra polinom zamanda çözülebilecektir. Bu konu ileride görülecektir.

> **Ders sonrası not:** Non-deterministik polinom (NP) sınıfı büyük olasılıkla "polinom olmayan" anlamına gelmez. Bu sınıf, bir çözümü polinom zamanda doğrulanabilen problemlerdir ve polinom zamanda çözülebilenler de büyük olasılıkla bu sınıfın içindedir.

25. Genellenebilirlik ve Ders Kitabı

Hoca kolay bir araştırma konusu olarak asal sayı bulma algoritmalarını önerdi: Bunlar karşılaştırılabilir veya yeni bir algoritma tasarlanabilir. Bir öğrencinin geçmişte "altının katından bir fazlaysa asaldır" dediğini, bunun ilk birkaç terimde tutsa da geçerli bir çözüm olmadığını anlattı.

Buradan iyi algoritma ölçütlerine bir madde daha ekledi: **Genellenebilirlik.** Bir probleme genel bir çözüm üreten algoritma, yalnız özel bir amaç için çalışan algoritmadan daha kaliteli ve değerlidir.

Soru: Dersin başındaki slaytta bir kitap vardı, origami örneği de ondandı. Hangi kitap?

Cevap: Hoca kitabı *Computer Science: An Overview* olarak tanıttı, 13. baskısı 2019'dur. Türkçesi de vardır, ama İngilizcesinin okunmasını tavsiye etti. Slaytlar yalnız hangi konunun anlatılacağını söyler, örneğin "primitifler" der ve hoca onu derste açar. Bu yüzden slayttan ders çalışılmaz, kitabın kendisi okunmalıdır.

> **Ders sonrası not:** Altyazıda yazar adı "Brockşehir" diye geçiyor. Kastedilen büyük olasılıkla J. Glenn Brookshear'dır.

Dersin Toparlaması

Dersin ilk yarısında bilgisayarın kullandığı 0 ve 1'leri nasıl okuyup temsil edeceğimizi gördük. İkilik sistemden sekizlik ve on altılık sisteme gruplamayla geçtik, gruplamaya her zaman sağdan başladık. Kesirli sayıları 2 ile çarparak çevirdik. İşaret bitinin neden en anlamlı bite konduğunu ve negatif sayıların ikiye tümlemeyle nasıl elde edildiğini öğrendik. Reel sayılar için IEEE 754 standardının işaret, exponent ve fraction alanlarını, bias değerini ve normalize edilmiş $1.xxx \times 2^y$ biçimini örneklerle çözdük.

İkinci yarıda algoritma kavramına geri döndük. Yol tarifi, turnike ve robota araba çizdirme örnekleri, her adımın tanımlı olması gerektiğini ve üç temel yapıdan başka bir yapı olmadığını gösterdi. İyi algoritmanın ölçütlerini ve aralarındaki ödünleşimi konuştuk. 1'den N'e toplam

örneği, doğru bir algoritmanın bile zaman ve bellek açısından çok daha iyi hale getirilebileceğini gösterdi. Hoca, sonraki derste iyi algoritma sorularının konuşulacağını ve en iyi ile en kötü durum analizlerinin ilerleyen derslerde ayrıntılandırılacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.